

Docker introduction

Presented by:

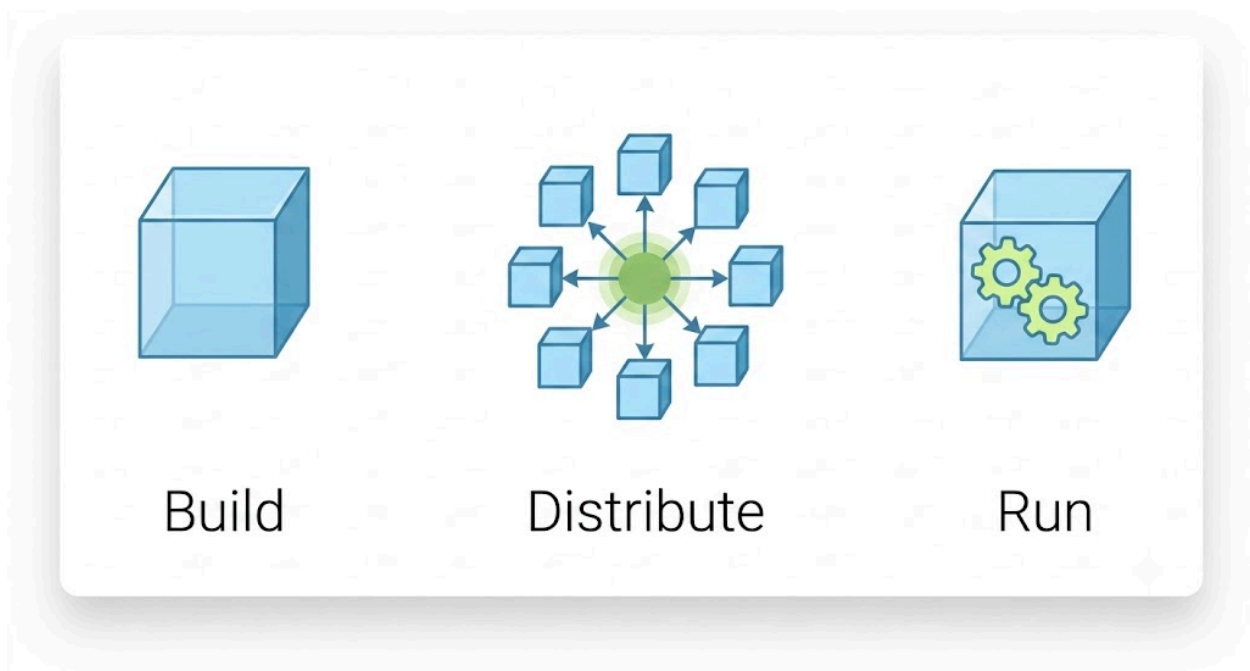
Cristian Camilo Cortes Ortiz

Table of contents

Problems in professional software development_____	3
Problems in software development_____	3
Problem with the software distribution_____	3
Problem when running software_____	3
VM issues_____	5
Containers_____	5
What is Docker?_____	6
Images_____	8
Layer system_____	9
Commands_____	10
docker_____	10
docker ps_____	10
docker ps -a_____	10
docker inspect c31e853d54af_____	10
docker inspect --format '{{.State.Pid}}' c31e853d54af_____	10
docker run_____	10
docker run -it_____	10
docker run --name containerName -d serviceName tail -f /dev/null_____	10
docker run --name containerName -d -p 8080:80 serviceName_____	10
docker run --name customName containerName_____	10
docker rename containerName newName_____	11
docker rm containerName_____	11
docker container prune_____	11
docker system prune_____	11
docker exec it containerName serviceName_____	11
docker stop containerName_____	11
docker stats_____	11
docker rm -f containerName_____	11
docker rm containerName_____	11
docker logs containerName_____	11
docker logs -f containerName_____	11
docker logs --tail Number -f containerName_____	12
exit_____	12
docker volume ls_____	12
docker volume create volumeName_____	12
docker run -d --name db --mount src=folderName,dst=/data/path serviceName_____	12
docker cp fileName containerName:/route/path/_____	12

docker containerName:/route/path/file localFilename_____	12
docker pull imageName:tagVersion_____	12
docker push userName/imageName:tagVersion_____	12
Docker tag imageName:tagVersion repositoryName/imageName:tagVersion_____	12
docker images_____	12
docker image —filter=reference='imageName:tagVersion'_____	13
docker login_____	13
docker history imageName_____	13
docker build -t -d dockerfileName._____	13
docker network ls_____	13
docker network create —attachable networkName_____	13
docker network inspect networkName_____	13
docker network connect networkName containerName_____	13
docker network prune_____	13
docker compose up_____	13
docker compose down_____	13
docker compose logs_____	14
docker compose logs serviceName_____	14
docker compose exec serviceName_____	14
docker rm -f \$(docker ps -a -q)_____	14
docker rmi -f \$(docker images -a -q)_____	14
docker volume prune -f_____	14
docker save imageName > newFileName_____	14
docker load —input fileName_____	14

Problems in professional software development



Problems in software development

- Dependencies
- Runtime Environment
- Equivalence to the Production Environment
- External Services

Problem with the software distribution

- Repository divergence
- Artifact divergence
- Versioning

Problem when running software

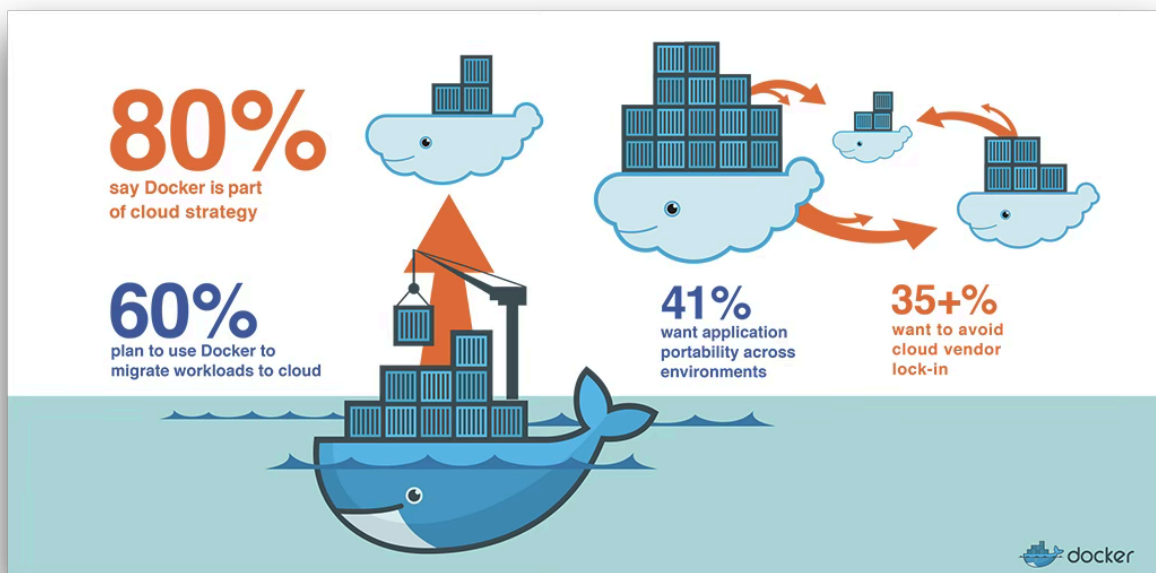
- Compatibility with the production environment
- Dependencies
- Availability of external services
- Hardware resources

“

**Docker enables you to *build*,
distribute, and *run* any
application anywhere.**

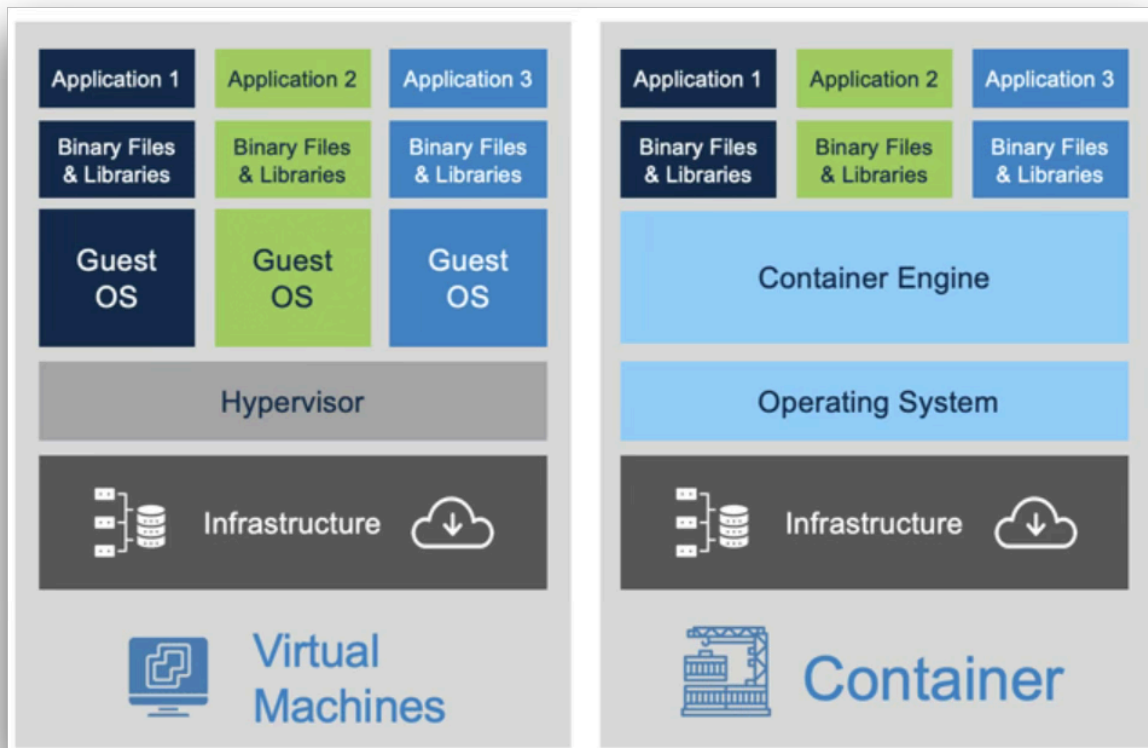
”

Official Docker Documentation



VM issues

- Weight
- Administrative costs
- Multiple formats



Containers

Containers are used to build and deploy software.

Features:

- Flexible
- Lightweight
- Portable
- Low coupling
- Scalable
- Secure

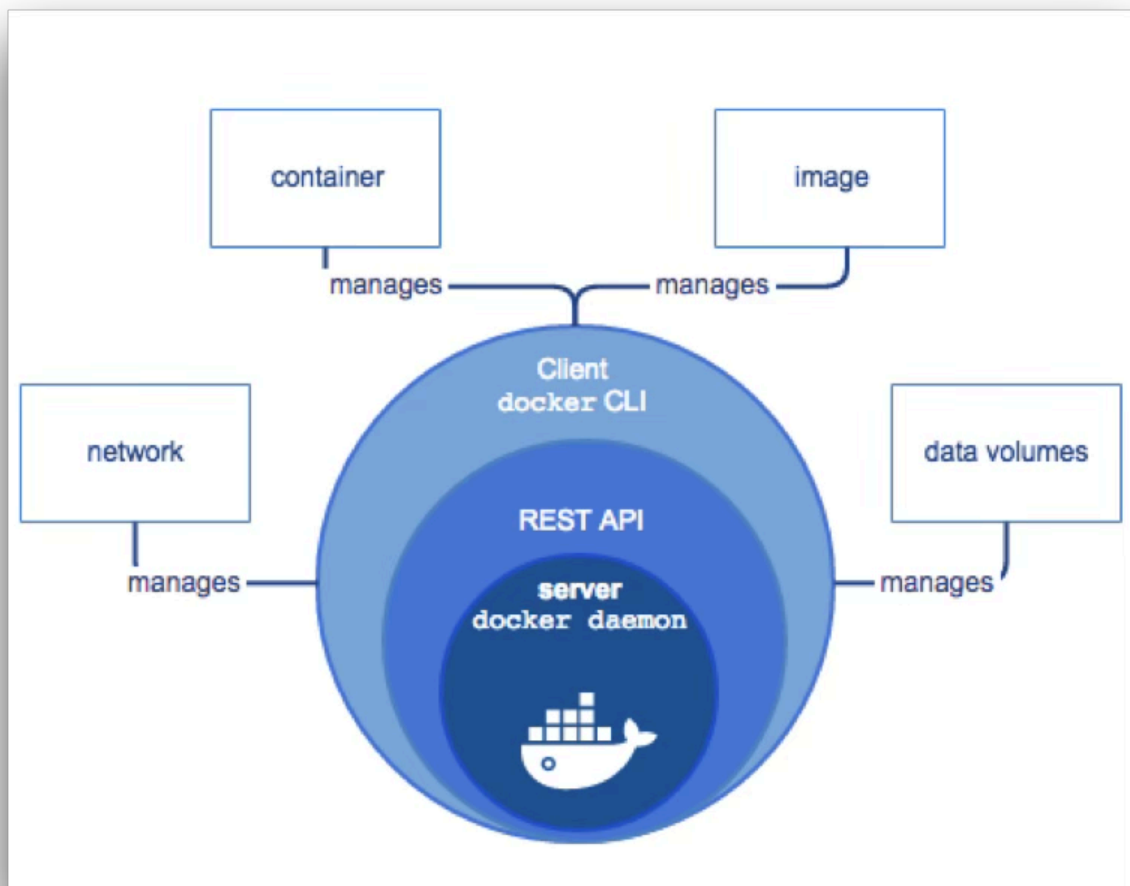
What is Docker?

Docker is a software platform that uses container technology to enable the efficient and portable packaging, distribution, and execution of applications across computing environments. Docker uses containers to encapsulate applications and their dependencies, including the libraries, files, and tools necessary for the application to run independently. Containers provide an isolated and consistent environment that ensures the application behaves the same way regardless of the runtime environment.

Docker allows you to package applications and their dependencies into container images. A container image is a lightweight, self-contained package that includes everything needed to run an application, from the code to the libraries and configuration files.

Docker allows you to package applications and their dependencies into container images. A container image is a lightweight, self-contained package that includes everything needed to run an application, from the code to the libraries and configuration files. Container images can be distributed through repositories, such as Docker Hub, where developers can share and download public and private images. This facilitates distribution and collaboration in application development.

Docker provides a consistent runtime environment for containers, enabling applications to run quickly and reliably in different environments, such as physical machines, virtual machines, on-premises environments, or in the cloud.



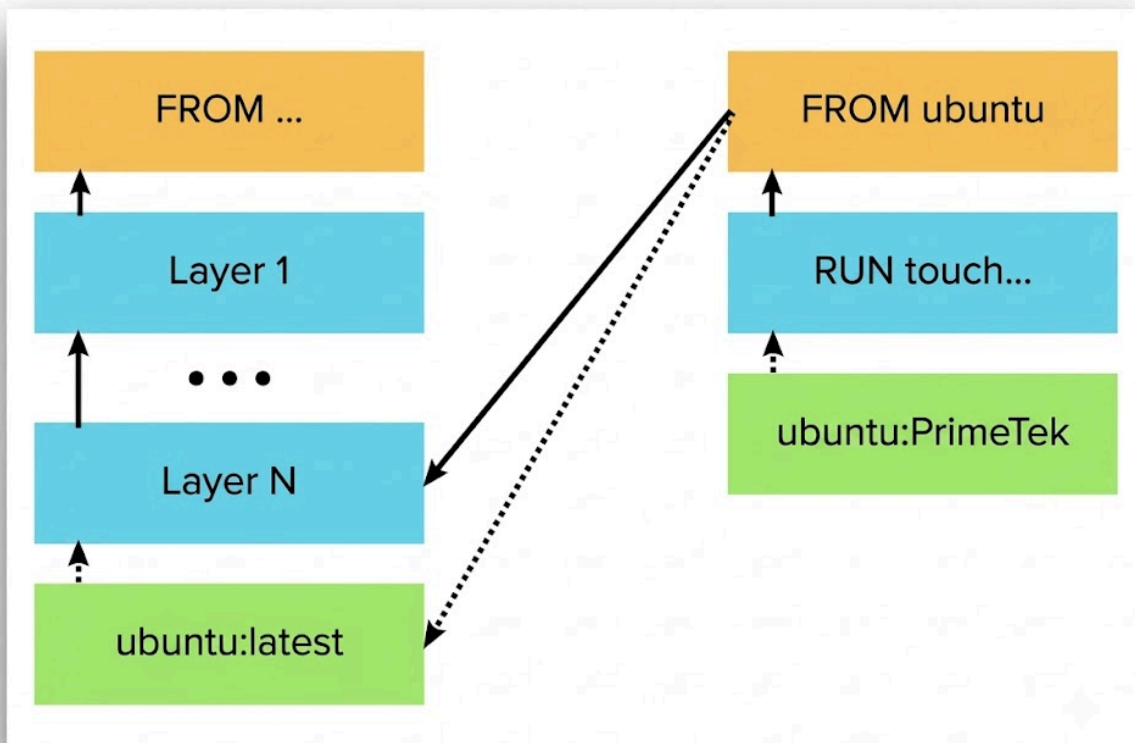
Images

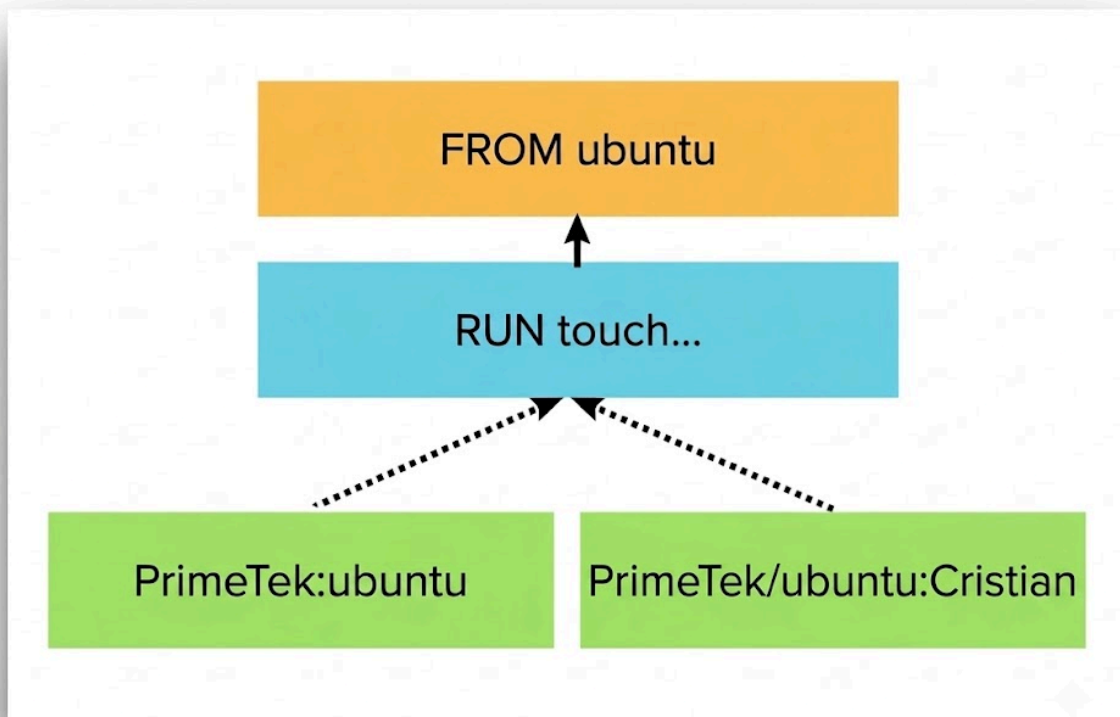
A Docker image is a lightweight, self-contained package that contains everything needed to run an application. Essentially, a Docker image encapsulates an application's runtime environment, including the code, libraries, dependencies, and necessary configurations.

A Docker image consists of one or more read-only layers. Each layer represents a change to the container's filesystem, such as installing a package, adding a file, or configuring a command. These layers are immutable, meaning they cannot be modified once created, but they can be shared and reused across different images.

Docker uses a layered file system to build images. Each layer represents a set of changes to the container's file system. When a container is created from an image, Docker efficiently combines these layers and provides a consistent view of the file system.

Docker images are defined using a simple, portable format that specifies the file system layers, metadata, and commands needed to set up the container. This allows Docker images to be easily shared, distributed, and run in different Docker environments, such as local machines, remote servers, or cloud environments.





Layer system

The Docker layer system is a fundamental component of its architecture that enables efficient management of container image storage and distribution. In Docker, each container image consists of a series of file system layers. These layers are immutable, read-only representations of the changes made to the container's file system. Each layer contains files, directories, and metadata that represent a specific state of the file system.

Commands

docker

Allows you to use the Docker API CLI.

docker ps

Allows you to view the containers that are running.

docker ps -a

Allows you to view all existing containers.

docker inspect c31e853d54af

Allows you to inspect a specific container using its ID.

docker inspect --format '{{.State.Pid}}' c31e853d54af

Allows you to view the process ID of the process currently running on the native machine.

docker run

Allows you to run a specific container.

docker run -it

Allows you to run a container in interactive mode.

docker run --name containerName -d serviceName tail -f /dev/null

Allows you to run a service inside a container in the background until something happens.

docker run --name containerName -d -p 8080:80 serviceName

The -p option allows you to specify the local machine's port and, followed by a colon, the virtual machine's (container's) port.

docker run --name customName containerName

Allows you to customize the name of a specific container.

docker rename containerName newName

Allows you to rename a specific container.

docker rm containerName

Allows you to delete a specific container by name.

docker container prune

Allows you to delete all stopped containers.

docker system prune

Allows you to delete all containers, networks, images, and builds that are paused.

docker exec it containerName serviceName

Allows you to run a command or process in an existing container that is already running.

docker stop containerName

Allows you to stop the running container.

docker stats

Allows you to view the hardware resource usage of a running container.

docker rm -f containerName

Allows you to delete a container, regardless of whether it is running or not.

docker rm containerName

Allows you to delete an existing container as long as it is stopped.

docker logs containerName

Allows you to view the logs for a container that has been run.

docker logs -f containerName

Allows you to view the logs that a container is generating in real time.

docker logs --tail Number -f containerName

Allows you to view a specific number of logs that a container is generating in real time.

exit

Allows you to exit the currently running process.

docker volume ls

Allows you to view the volumes within Docker.

docker volume create volumeName

Allows you to create a volume within Docker.

**docker run -d --name db --mount src=folderName,dst=/data/path
serviceName**

Allows you to create a container and add a volume to it.

docker cp fileName containerName:/route/path/

Allows you to copy a file or directory inside a Docker container.

docker containerName:/route/path/file localFilename

Allows you to copy a file or directory located inside a container to the local machine, assigning a name to this new file or directory.

docker pull imageName:tagVersion

Allows you to pull an image from the Docker Hub repository in a specific version.

docker push userName/imageName:tagVersion

Allows you to push an image to the Docker Hub repository as a specific version.

**Docker tag imageName:tagVersion
repositoryName/imageName:tagVersion**

Allows you to push an image to the Docker Hub repository as a specific version.

docker images

Allows you to view images within Docker.

docker image --filter=reference='imageName:tagVersion'

Allows you to filter Docker images by name and version.

docker login

Allows you to authenticate your Docker account credentials using the Docker CLI in the command terminal.

docker history imageName

Allows you to view the layers that make up an image.

docker build -t -d dockerfileName.

Allows you to build an image based on a Dockerfile.

docker network ls

Allows you to view network interfaces within Docker.

docker network create --attachable networkName

Allows you to create a network within Docker.

docker network inspect networkName

Allows you to inspect a specific network using its ID.

docker network connect networkName containerName

Allows you to connect a container to a Docker network.

docker network prune

Allows you to delete all Docker networks.

docker compose up

Allows services to be restored.

docker compose down

Allows suspending services.

docker compose logs

Allows you to view the logs for all services.

docker compose logs serviceName

Allows you to view the logs for a specific service.

docker compose exec serviceName

Allows you to run a service.

docker rm -f \$(docker ps -a -q)

Allows you to delete all Docker containers.

docker rmi -f \$(docker images -a -q)

Allows you to delete all Docker images.

docker volume prune -f

Allows you to delete all Docker volumes.

docker save imageName > newFileName

Allows you to export a Docker image to a compressed file.

docker load --input fileName

Allows you to import an image into Docker from a compressed file.